

DATABASE MANAGEMENT SYSTEM LAB

Name: Aditi M

USN: 1RUA25BCA0006

Subject: Database Management System

LAB 7

Question: Implement different types of Joins: Implement various types of joins in SQL, including Inner Join, Outer Join, and Natural Join, with suitable examples.

Output:

The screenshot displays the OneCompiler MySQL interface with two queries and their results.

Query 1: A series of SQL statements to create and populate tables.

```
1 DROP TABLE IF EXISTS employee_log;
2 DROP TABLE IF EXISTS departments;
3 DROP TABLE IF EXISTS contractors;
4 DROP TABLE IF EXISTS employees;
5
6 CREATE TABLE employees (
7     emp_id INT PRIMARY KEY,
8     emp_name VARCHAR(50),
9     department VARCHAR(30),
10    salary DECIMAL(10,2),
11    bonus DECIMAL(10,2),
12    hire_date DATE,
13    email VARCHAR(50)
14 );
15
16 INSERT INTO employees VALUES (101, 'Aditi Rao', 'Analytics', 55000.00, 5000.00, '2023-06-15', 'aditigcorp.com');
17 INSERT INTO employees VALUES (102, 'Ravi Kumar', 'Finance', 62000.50, NULL, '2022-01-18', 'ravi@corp.com');
18 INSERT INTO employees VALUES (103, 'Sneha Iyer', 'Analytics', 48000.75, 3000.00, '2024-03-01', 'sneha@corp.com');
19 INSERT INTO employees VALUES (104, 'Arjun Mehta', 'HR', 41000.00, NULL, '2021-11-20', NULL);
20 INSERT INTO employees VALUES (105, 'Priya Nair', 'Finance', 70000.00, 8000.00, '2020-08-05', 'priya@corp.com');
21
22 CREATE TABLE contractors (
23     con_id INT PRIMARY KEY,
24     con_name VARCHAR(50),
25     department VARCHAR(30),
26     fee DECIMAL(10,2)
27 );
28
29 INSERT INTO contractors VALUES (201, 'Neha Singh', 'Analytics', 45000.00);
30 INSERT INTO contractors VALUES (202, 'Ravi Kumar', 'Finance', 62000.50);
31 INSERT INTO contractors VALUES (203, 'Omair Khan', 'IT', 52000.00);
32
33 CREATE TABLE departments (
34     department VARCHAR(30) PRIMARY KEY,
35     location VARCHAR(30)
36 );
37
38 INSERT INTO departments VALUES ('Analytics', 'Bengaluru');
39 INSERT INTO departments VALUES ('Finance', 'Mumbai');
40 INSERT INTO departments VALUES ('HR', 'Delhi');
41 INSERT INTO departments VALUES ('Marketing', 'Chennai');
```

Query 2: SQL statements for joins and a natural join.

```
33 CREATE TABLE employee_log (
34     log_id INT AUTO_INCREMENT PRIMARY KEY,
35     emp_id INT,
36     action_type VARCHAR(20),
37     description VARCHAR(100)
38 );
39
40 SELECT employees.emp_name, employees.department, departments.location
41 FROM employees
42 INNER JOIN departments ON employees.department = departments.department;
43
44 SELECT contractors.con_name, contractors.department, departments.location
45 FROM contractors
46 LEFT JOIN departments ON contractors.department = departments.department;
47
48 SELECT contractors.con_name, departments.department, departments.location
49 FROM contractors
50 RIGHT JOIN departments ON contractors.department = departments.department;
51
52 SELECT contractors.con_name, departments.department, departments.location
53 FROM contractors
54 UNION
55 SELECT contractors.con_name, departments.department, departments.location
56 FROM contractors
57 RIGHT JOIN departments ON contractors.department = departments.department;
58
59 SELECT * FROM contractors NATURAL JOIN departments;
```

Outputs:

Query 1 Output:

emp_name	department	location
Aditi Rao	Analytics	Bengaluru
Ravi Kumar	Finance	Mumbai
Sneha Iyer	Analytics	Bengaluru
Arjun Mehta	HR	Delhi
Priya Nair	Finance	Mumbai

con_name	department	location
Neha Singh	Analytics	Bengaluru
Ravi Kumar	Finance	Mumbai
Omair Khan	IT	Delhi
NULL	HR	Delhi
NULL	Marketing	Chennai

Query 2 Output:

emp_name	department	location
Sneha Iyer	Analytics	Bengaluru
Arjun Mehta	HR	Delhi
Priya Nair	Finance	Mumbai

con_name	department	location
Neha Singh	Analytics	Bengaluru
Ravi Kumar	Finance	Mumbai
Omair Khan	IT	Delhi
NULL	HR	Delhi
NULL	Marketing	Chennai

con_name	department	location
Neha Singh	Analytics	Bengaluru
Ravi Kumar	Finance	Mumbai
Omair Khan	IT	Delhi
NULL	HR	Delhi
NULL	Marketing	Chennai

department	con_id	con_name	fee	location
Analytics	201	Neha Singh	45000.00	Bengaluru
Finance	202	Ravi Kumar	62000.50	Mumbai

LAB 8

Question: Implement Grouping and Ordering: Study and implement the GROUP BY and HAVING clauses, ORDER BY clause.

Output:

The screenshot displays the OneCompiler MySQL IDE interface. The left pane contains a list of SQL queries, and the right pane shows the output of the selected query (query 93).

Query 93:

```
SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department HAVING SUM(salary) > 50000 ORDER BY total_salary DESC;
```

Output:

department	total_salary
Finance	132000.50
Analytics	103000.75

The output shows the total salary for each department, filtered by a HAVING clause (SUM(salary) > 50000) and ordered by total salary in descending order.

LAB 9

Question: Implement Procedure. (Basic procedure, procedure for Insert, Update and Delete)

Output:

The screenshot shows the OneCompiler MySQL interface. The left pane contains SQL queries for Lab 9, and the right pane shows the output of the executed queries.

```
66 SELECT contractors.con_name, departments.department, departments.location
67 FROM contractors
68 LEFT JOIN departments ON contractors.department = departments.department
69 UNION
70 SELECT contractors.con_name, departments.department, departments.location
71 FROM contractors
72 RIGHT JOIN departments ON contractors.department = departments.department;
73
74
75 SELECT * FROM contractors NATURAL JOIN departments;
76
77
78 SELECT department, COUNT(*) AS total_employees FROM employees GROUP BY department;
79
80 SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department;
81
82 SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department;
83
84 SELECT department, COUNT(*) AS total_employees FROM employees GROUP BY department HAVING COUNT(*) > 1;
85
86 SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department HAVING AVG(salary) > 50000;
87
88 SELECT * FROM employees ORDER BY salary ASC;
89
90 SELECT * FROM employees ORDER BY salary DESC;
91
92 SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department HAVING SUM(salary) > 50000 ORDER BY total_salary DESC;
93
94
95 DROP PROCEDURE IF EXISTS show_employees;
96 DROP PROCEDURE IF EXISTS insert_employee;
97 DROP PROCEDURE IF EXISTS update_salary;
98 DROP PROCEDURE IF EXISTS delete_employee;
99
100 CREATE PROCEDURE show_employees() SELECT * FROM employees;
101
102 CREATE PROCEDURE insert_employee(IN p_id INT, IN p_name VARCHAR(50), IN p_dept VARCHAR(30), IN p_salary DECIMAL(10,2)) INSERT INTO employees
103
104 CREATE PROCEDURE update_salary(IN p_id INT, IN p_salary DECIMAL(10,2)) UPDATE employees SET salary = p_salary WHERE emp_id = p_id;
105
106 CREATE PROCEDURE delete_employee(IN p_id INT) DELETE FROM employees WHERE emp_id = p_id;
107
```

The output on the right shows the results of the queries, including department counts, average salaries, and a list of employees with their details.

LAB 10

Question: Implement Triggers (validation before and after insert, delete and update)

Output:

The screenshot shows the OneCompiler MySQL interface. The left pane contains SQL queries for Lab 10, and the right pane shows the output of the executed queries.

```
92
93 SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department HAVING SUM(salary) > 50000 ORDER BY total_salary DESC;
94
95
96 DROP PROCEDURE IF EXISTS show_employees;
97 DROP PROCEDURE IF EXISTS insert_employee;
98 DROP PROCEDURE IF EXISTS update_salary;
99 DROP PROCEDURE IF EXISTS delete_employee;
100
101 CREATE PROCEDURE show_employees() SELECT * FROM employees;
102
103 CREATE PROCEDURE insert_employee(IN p_id INT, IN p_name VARCHAR(50), IN p_dept VARCHAR(30), IN p_salary DECIMAL(10,2)) INSERT INTO employees (emp_id, emp_name, department, salary, hire_date) VALUES (p_id, p_name, p_dept, p_salary, NOW());
104
105 CREATE PROCEDURE update_salary(IN p_id INT, IN p_salary DECIMAL(10,2)) UPDATE employees SET salary = p_salary WHERE emp_id = p_id;
106
107 CREATE PROCEDURE delete_employee(IN p_id INT) DELETE FROM employees WHERE emp_id = p_id;
108
109
110 DROP TRIGGER IF EXISTS before_emp_insert;
111 DROP TRIGGER IF EXISTS after_emp_insert;
112 DROP TRIGGER IF EXISTS before_emp_update;
113 DROP TRIGGER IF EXISTS after_emp_update;
114 DROP TRIGGER IF EXISTS before_emp_delete;
115 DROP TRIGGER IF EXISTS after_emp_delete;
116
117
118 CREATE TRIGGER before_emp_insert BEFORE INSERT ON employees FOR EACH ROW SET NEW.salary = IF(NEW.salary < 0, 0, NEW.salary);
119
120
121 CREATE TRIGGER after_emp_insert AFTER INSERT ON employees FOR EACH ROW INSERT INTO employee_log (emp_id, action_type, description) VALUES (NEW.emp_id, 'INSERT', CONCAT('Added ', NEW.emp_name, ' to ', NEW.department));
122
123
124 CREATE TRIGGER before_emp_update BEFORE UPDATE ON employees FOR EACH ROW SET NEW.salary = IF(NEW.salary < OLD.salary, OLD.salary, NEW.salary);
125
126
127 CREATE TRIGGER after_emp_update AFTER UPDATE ON employees FOR EACH ROW INSERT INTO employee_log (emp_id, action_type, description) VALUES (NEW.emp_id, 'UPDATE', CONCAT('Salary ', OLD.salary, ' to ', NEW.salary));
128
129
130 CREATE TRIGGER before_emp_delete BEFORE DELETE ON employees FOR EACH ROW INSERT INTO employee_log (emp_id, action_type, description) VALUES (OLD.emp_id, 'BEFORE DELETE', CONCAT('About to delete ', OLD.emp_name));
131
132
133 CREATE TRIGGER after_emp_delete AFTER DELETE ON employees FOR EACH ROW INSERT INTO employee_log (emp_id, action_type, description) VALUES (OLD.emp_id, 'AFTER DELETE', CONCAT('Deleted ', OLD.emp_name));
134
```

The output on the right shows the results of the queries, including department counts, average salaries, and a list of employees with their details.